

시냅스 1차 팀 프로젝트 A팀(박소현, 이지현, 최유경)

1. 데이터 이해

train.csv	모델 학습을 위한 메타데이터로 train 이미지에 대한 이미지 명과 정답 label 을 매핑하는 역할	<pre> file_name label 0 001.PNG 9 1 002.PNG 4 2 003.PNG 1 3 004.PNG 1 4 005.PNG 6 </pre>
test.csv	모델 평가 및 결과 제출용 메타데이터. Test 이미지의 file name 만 포함	<pre> -----test data----- file_name 0 001.PNG 1 002.PNG 2 003.PNG 3 004.PNG 4 005.PNG </pre>
train 이미지와 test 이미지의 차이	train 이미지는 모델이 특징을 추출하고 가중치를 업데이트하는 데 직접적으로 사용되는 훈련용 데이터. test 이미지와 달리 정답이 명확히 제공.	
클래스 개수	10 (0~9 인덱스)	<pre> label 0 67 1 71 2 75 3 71 4 67 5 75 6 75 7 72 8 74 9 76 Name: count, dtype: int64 </pre>
데이터 개수	Train Data : 723개, Test Data: 199개	<pre> RangeIndex: 723 entries, 0 to 722 Data columns (total 2 columns): RangeIndex: 199 entries, 0 to 198 </pre>

이미지 크기 및 색상 채널 처리 방식	원본 이미지의 shape는 960*540*3 (W,H,C) CNN 모델은 동일한 이미지 입력을 요하고, Transfer learning 시 Pretrained 된 모델이 224*224 크기의 imageNet 데이터를 사용했으므로 비교를 위해 224*224해상도로 resize 했다.	<pre># 이미지 형식 확인 import cv2 image_path = '/content/drive/MyDrive/Synap img = cv2.imread(image_path, cv2.IMREAD_UN height, width, channels = img.shape print(f"이미지 크기: 너비={width}, 높이={h 이미지 크기: 너비=960, 높이=540, 채널=3</pre>
-----------------------------	---	---

2. 전체 실행 흐름 설명

1) 데이터 불러오기

모델 학습을 시작하기 전, 어떤 데이터가 있는지 파악하고, **train.csv**, **test.csv**와 같은 메타데이터를 메모리에 로드하여, 정답(**Label**) 분포와 파일 식별자 정보를 확인하기 위해 필요하다.

2) 이미지 경로 생성

코드가 **Storage**에서 실제 이미지 파일을 찾아 읽어오려면, 데이터가 저장된 기본 폴더 위치와 파일명을 결합하여 완전한 절대 경로 또는 상대 경로를 만들어야 한다.

3) Dataset 구성

디스크에 있는 개별 이미지 파일을 딥러닝 프레임워크가 이해할 수 있는 수학적 형태인 텐서(**Tensor**)로 변환 하기 위해 필요하다. **resize**나 증감 적용, **normalization** 등등 모델이 학습하기 좋은 형태로 데이터를 가공한다.

4) DataLoader 구성

Data를 **batch**라는 묶음으로 나눠 모델에 **data**를 **load** 시키는 역할을 한다.. **shuffle** 하는 등 데이터를 제공 한다.

5) 모델 정의

입력 된 이미지에서 특징을 추출하고 **classification**하는 역할을 한다.

6) Loss/Optimizer 설정

모델의 예측값과 실제 정답이 얼마나 차이가 나는지 오차를 수치화하는 **Loss Function**을 정하고, 이 **Loss**를 줄이기 위한 방향이나 얼마나 **w**를 조정할 지 정하는 **optimization**을 설정한다.

7) 학습

DataLoader가 던져주는 배치 데이터를 모델이 예측하고, **Loss**를 계산한 뒤, 오차를 역전파하여 **Optimizer**가 가중치를 업데이트하는 과정을 지정된 에폭 수 만큼 반복한다. 이 과정을 통해 모델은 이미지의 특징과 정답 사이의 패턴을 스스로 찾아내게 된다.

8) 검증

학습된 모델이 과적합(**Overfitting**) 상태에 빠지지 않았는지 확인 위해 필요하다. 학습에 한 번도 사용되지 않은 검증 데이터(**Validation Set**)를 모델에 넣어 성능을 평가하며, 이때는 가중치 업데이트를 하지 않습니다. 이 지표를 바탕으로 가장 성능이 좋은 시점의 모델(**Best Model**)을 저장한다.

9) 테스트 예측

완전히 학습이 끝난 최적의 모델을 사용하여 정답이 없는 데이터에 대한 최종 클래스를 추론(Inference)하는 단계이며, 실제 모델의 실전 능력을 평가하기 위한 단계이다.

10) 제출 파일 생성

모델 능력 평가를 위한 제출 파일을 생성하는 단계이다.

3. 기본 CNN 모델 설명

1) CNN 구조

3개의 BasicBlock으로 이루어진 특징 추출기(Feature Extractor)와 3개의 Linear 레이어로 이루어진 분류기(Classifier)로 구성했다. 네트워크가 깊어질수록 채널 수(채널/차원)는 증가(3 → 32 → 128 → 256)하여 고수준의 복잡한 특징을 학습하고, 공간 해상도는 줄어드는 전형적인 피라미드 형태이다.

2) Conv2d의 역할

입력 이미지 또는 이전 층의 Feature Map에서 시각적 패턴을 추출함. 학습 가능한 가중치를 가진 필터(Filter/Kernel)가 이미지를 슬라이딩하며 합성곱 연산을 수행하여 Local Features를 얻는다.

3) ReLU의 역할

네트워크에 비선형성을 부여한다. feature의 복잡한 패턴을 학습 할 수 있게 하고, 깊은 신경망에서 기울기 소실 문제를 완화한다.

4) MaxPool2d의 역할

해상도를 줄임으로써 더 넓은 수용역-역에서 전역적인 feature를 얻어낸다. 특징 맵의 크기를 줄여, 파라미터수와 연산량을 감소시키고, 과적합을 방지한다. 객체의 미세한 이동이나 왜곡에 강건성을 갖도록 다.

5) Flatten이 필요한 이유

다차원 텐서를 1차원 벡터로 변화시켜, 추출된 특징을 기반으로 최종 분류를 수행하기 위해 FC layer에 입력으로 넣기 위해서다.

6) 마지막 Linear layer 출력 크기를 10으로 설정한 이유

본 모델의 목적이 주어진 이미지를 10개의 범주로 분류하는 Multiclass Classification이기 때문이다. 출력된 10개의 노드 값이 각각 0~9까지의 인덱스에 대응되며, 최종적으로는 이미지가 어떤 class에 속할 확률로 변화 된다.

4. 전이학습 모델 설명

1) 선택한 모델

ResNet18

2) 모델 선택 이유

ResNet은 잔차 연결(residual connection) 기법을 사용하여 층이 깊어져도 기울기 소실 문제가 발생하지 않도록 설계된 검증된 아키텍처이다. 특히 18개 층을 가진 ResNet18은 구조가 비교적 가벼워 연산 속도가 빠르며, Colab 환경인 T4 GPU에서 10개의 클래스를 분류하는 기본적인 이미지 분류 작업에 효율적이면서도 강력한 성능을 발휘한다.

3) classifier layer 수정 방식

ResNet18의 마지막 fully connected layer를 현재 데이터셋의 클래스 개수인 10개에 맞게 조정하였다. 이때 기존 계층의 입력 노드 수(in_features)는 그대로 가져오고, 출력 노드를 분류할 클래스 개수인 10으로 변경하여 새로운 nn.Linear 계층을 정의하였다.

4) pretrained 사용 여부

ImageNet 데이터셋으로 훈련된 가중치(ResNet18)를 불러와 초기 가중치로 사용하여 데이터 전처리 단계에서도 ImageNet 데이터셋의 평균 및 표준편차 수치를 그대로 사용하여 정규화를 수행하였다.

코드 근거:

- 모델 가중치 로드: `transfer_model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)`
- 정규화 파라미터: `mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]`

5) freeze 전략

특징을 추출하는 합성곱 계층들의 가중치를 고정하지 않고 모든 파라미터가 갱신되도록 열어두었다.

코드 근거: 모델의 모든 파라미터를 optimizer에 전달할 때 (`optimizer = torch.optim.Adam(transfer_model.fc.parameters(), ...)`)을 사용하지 않음

- `optimizer = torch.optim.Adam(transfer_model.parameters(), ...)`

6) fine-tuning 방식

전체 파라미터 미세 조정(End-to-End Fine-tuning) 방식을 사용하여 모델 전체 계층의 가중치를 새로운 데이터셋에 맞게 업데이트하는 방식을 사용하였다. 이미 잘 학습된 기존 가중치가 무너지지 않으면서 천천히 최적화될 수 있도록, 학습률을 `1e-4(0.0001)`로 설정하였다.

코드 근거:

- `optimizer = torch.optim.Adam(..., lr=1e-4)`

7) 전이학습의 장단점

장점:

- 학습 속도 및 성능 향상: 방대한 데이터로 이미 윤곽선, 질감 등의 특징을 추출하는 방법을 알고 있는 상태에서 시작하므로, 바닥부터 학습하는 것보다 훨씬 적은 에포크 만으로도 높은 정확도에 도달한다. 실제 코드 실행 결과에서도 **Epoch 2**만에 검증 정확도(**Val Acc**)가 **0.98**을 넘는 것을 볼 수 있다.
- 데이터 부족 문제 해결: 모델이 이미 일반적인 시각적 특징을 학습해 두었기 때문에, 훈련 데이터가 상대적으로 적은 상황에서도 과적합에 덜 취약하다.

단점:

- 도메인 불일치 한계: 원본 모델이 학습한 데이터(e.g. 일반 사물, 동물 등)와 새롭게 학습할 데이터(e.g. 흑백 의료 영상 등)의 시각적 특성이 완전히 다를 경우, 전이학습의 효과가 떨어지거나 오히려 악영향을 줄 수 있다.
- 구조적 제약: **ResNet18**은 설계될 때 가로세로 **224x224** 픽셀 크기의 **RGB 3채널** 컬러 사진만 입력받도록 통로가 고정되어 만들어졌다. 이때 사전 학습 모델이 요구하는 입력 이미지의 규격을 강제로 맞춰주어야 하므로, 전처리 과정이 고정된다는 제약이 있다.

5. 실험 결과 비교

**** colab 환경에서 학습 실행시, 매번 데이터가 shuffle되어 학습되므로 같은 모델이더라도 학습마다 Best Val Acc가 상이 해진다. 따라서 다음과 같이 seed가 고정되도록 코드를 작성했다.**

```
1 import random
2 import numpy as np
3
4 def set_seed(seed=42):
5     random.seed(seed)
6     np.random.seed(seed)
7     torch.manual_seed(seed)
8     torch.cuda.manual_seed(seed)
9     torch.cuda.manual_seed_all(seed)
10    torch.backends.cudnn.deterministic = True
11    torch.backends.cudnn.benchmark = False
12
13 set_seed(42)
```

실험 결과 비교(하이퍼 파라미터 조정) 15epoch

```

lr batch_size optimizer best_val_acc
4 0.0001 16 Adam 0.820690
1 0.0010 16 AdamW 0.813793
6 0.0001 32 Adam 0.800000
3 0.0010 32 AdamW 0.793103
7 0.0001 32 AdamW 0.793103
5 0.0001 16 AdamW 0.786207
2 0.0010 32 Adam 0.779310
0 0.0010 16 Adam 0.751724

```

실험 번호	모델	pretrained	freeze 전략	Optimizer	LR	Batch Size	Augmentation	Best Val Acc
0	baseline(15)	x	x	Adam	1e-3	16	x	0.7517
0	hpram0	x	x	Adam	1e-3	16	x	0.7517
1	hpram1	x	x	AdamW	1e-3	16	x	0.8138
2	hpram2	x	x	Adam	1e-3	32	x	0.7793
3	hpram3	x	x	AdamW	1e-3	32	x	0.7931
4	hpram4	x	x	Adam	1e-4	16	x	0.8207
5	hpram5	x	x	AdamW	1e-4	16	x	0.7862
6	hpram6	x	x	Adam	1e-4	32	x	0.8000
7	hpram7	x	x	AdamW	1e-4	32	x	0.7931

실험 결과 비교(데이터 증강) 30 epoch

실험 번호	모델	pretrained	freeze 전략	Optimizer	LR	Batch Size	Augmentation	Best Val Acc
8	baseline(30)	x	x	Adam	1e-3	16	x	0.8414
9	0	x	x	Adam	1e-3	16	randomHorizontalFlip, RandomRotation(degree : 30)	0.8138

실험 결과 비교(전이 학습) 15 epoch

```
Epoch [1/15]: 100%|██████████| 37/37 [02:26<00:00, 3.96s/it, train_acc=0.8097, train_loss=0.7400]
Epoch [1/15] | Train Loss: 0.7400 | Train Acc: 0.8097 | Val Loss: 0.0902 | Val Acc: 0.9862
Epoch [2/15]: 100%|██████████| 37/37 [00:14<00:00, 2.57it/s, train_acc=0.9965, train_loss=0.0497]
Epoch [2/15] | Train Loss: 0.0497 | Train Acc: 0.9965 | Val Loss: 0.0285 | Val Acc: 1.0000
Epoch [3/15]: 100%|██████████| 37/37 [00:14<00:00, 2.52it/s, train_acc=1.0000, train_loss=0.0201]
Epoch [3/15] | Train Loss: 0.0201 | Train Acc: 1.0000 | Val Loss: 0.0223 | Val Acc: 1.0000
Epoch [4/15]: 100%|██████████| 37/37 [00:13<00:00, 2.76it/s, train_acc=1.0000, train_loss=0.0152]
Epoch [4/15] | Train Loss: 0.0152 | Train Acc: 1.0000 | Val Loss: 0.0205 | Val Acc: 1.0000
Epoch [5/15]: 100%|██████████| 37/37 [00:13<00:00, 2.72it/s, train_acc=1.0000, train_loss=0.0158]
Epoch [5/15] | Train Loss: 0.0158 | Train Acc: 1.0000 | Val Loss: 0.0158 | Val Acc: 1.0000
Epoch [6/15]: 100%|██████████| 37/37 [00:13<00:00, 2.75it/s, train_acc=1.0000, train_loss=0.0081]
Epoch [6/15] | Train Loss: 0.0081 | Train Acc: 1.0000 | Val Loss: 0.0179 | Val Acc: 1.0000
Epoch [7/15]: 100%|██████████| 37/37 [00:12<00:00, 2.98it/s, train_acc=0.9965, train_loss=0.0154]
Epoch [7/15] | Train Loss: 0.0154 | Train Acc: 0.9965 | Val Loss: 0.0135 | Val Acc: 1.0000
Epoch [8/15]: 100%|██████████| 37/37 [00:12<00:00, 2.85it/s, train_acc=1.0000, train_loss=0.0080]
Epoch [8/15] | Train Loss: 0.0080 | Train Acc: 1.0000 | Val Loss: 0.0113 | Val Acc: 1.0000
Epoch [9/15]: 100%|██████████| 37/37 [00:13<00:00, 2.72it/s, train_acc=0.9983, train_loss=0.0200]
Epoch [9/15] | Train Loss: 0.0200 | Train Acc: 0.9983 | Val Loss: 0.0097 | Val Acc: 1.0000
Epoch [10/15]: 100%|██████████| 37/37 [00:13<00:00, 2.73it/s, train_acc=0.9983, train_loss=0.0174]
Epoch [10/15] | Train Loss: 0.0174 | Train Acc: 0.9983 | Val Loss: 0.0192 | Val Acc: 1.0000
Epoch [11/15]: 100%|██████████| 37/37 [00:13<00:00, 2.75it/s, train_acc=1.0000, train_loss=0.0077]
Epoch [11/15] | Train Loss: 0.0077 | Train Acc: 1.0000 | Val Loss: 0.0137 | Val Acc: 1.0000
Epoch [12/15]: 100%|██████████| 37/37 [00:13<00:00, 2.73it/s, train_acc=0.9983, train_loss=0.0070]
Epoch [12/15] | Train Loss: 0.0070 | Train Acc: 0.9983 | Val Loss: 0.0126 | Val Acc: 1.0000
Epoch [13/15]: 100%|██████████| 37/37 [00:13<00:00, 2.74it/s, train_acc=0.9965, train_loss=0.0165]
Epoch [13/15] | Train Loss: 0.0165 | Train Acc: 0.9965 | Val Loss: 0.0100 | Val Acc: 1.0000
Epoch [14/15]: 100%|██████████| 37/37 [00:13<00:00, 2.71it/s, train_acc=0.9983, train_loss=0.0186]
Epoch [14/15] | Train Loss: 0.0186 | Train Acc: 0.9983 | Val Loss: 0.0133 | Val Acc: 0.9931
Epoch [15/15]: 100%|██████████| 37/37 [00:12<00:00, 3.02it/s, train_acc=1.0000, train_loss=0.0076]
Epoch [15/15] | Train Loss: 0.0076 | Train Acc: 1.0000 | Val Loss: 0.0149 | Val Acc: 0.9931
Best Validation Accuracy: 1.0000
```

실험 번호	모델	pretrained	freeze 전략	Optimizer	L R	Batch Size	Augmentation	Best Val Acc
0	baseline	x	x	Adam	1e-3	16	x	0.7517
10	transfer	resNet 18	x	Adam	1e-4	16	x	1.000

6. 결과 해석

1) 어떤 모델의 성능이 가장 높았는가?

에포크 15라는 동일 조건에서 학습률(lr), 옵티마이저, 배치 사이즈를 일부 변경해 진행한 실험에서 가장 성능이 좋은 모델의 **Best Val Acc**는 **0.8207**에 그쳤으나 전이 학습 모델은 **1.0**을 기록하며 타 모델 대비 압도적인 성능을 보였다. 이는 에포크 30으로 진행된 다른 모델들의 결과와 견주어 보아도, 모든 검증 데이터를 오차 없이 완벽하게 분류해 낸 가장 우수한 성과이다.

2) 성능 차이가 발생한 이유는 무엇인가?

성능 차이가 발생한 주된 이유는 모델의 초기 가중치 상태, 아키텍처의 구조적 깊이, 그리고 미세 조정 전략 차이에 있다. 구체적인 요인은 다음과 같다.

- 가장 핵심적인 차이는 가중치의 초기화 상태이다. **Baseline** 모델은 무작위로 초기화된 상태에서 시작하여 선, 색상, 질감 등 기초적인 시각적 특징부터 새롭게 학습해야 했던 반면, 전이학습 모델은 방대한 **ImageNet** 데이터셋으로 이미 특징 추출 능력을 입증받은 **ResNet18**의 가중치를 초기값으로 사용했다. 즉, 사물의 특징을 파악하는 데 최적화된 상태에서 학습을 시작했기 때문에, 단 **5 Epoch** 만에 모든 검증 데이터를 완벽하게 분류해 내는 압도적인 성능에 도달할 수 있었다.
- 단순한 구조의 **Baseline** 모델과 달리, **ResNet18**은 잔차 연결 구조를 기반으로 설계된 깊은 신경망이다. 이 구조는 신경망의 층이 깊어질수록 학습 능력이 저하되는 기울기 소실

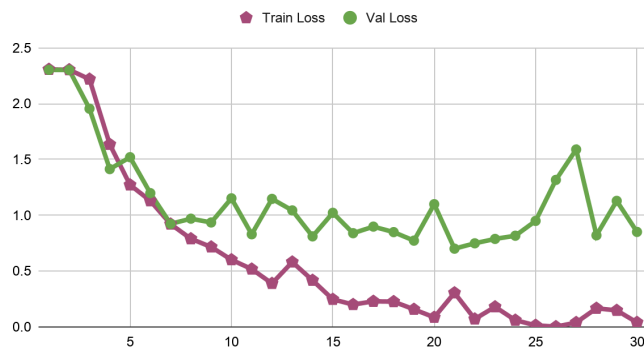
문제를 효과적으로 방지했으므로, **Baseline** 모델 대비 훨씬 더 복잡하고 고차원적인 이미지 특징을 손실 없이 정확하게 학습할 수 있었다.

- 제한된 훈련 데이터 환경에서 바닥부터 학습을 진행한 **Baseline** 모델은 노이즈나 쓸데없는 배경까지 전부 학습해버리는 과적합에 빠지기 쉽고, 특징 학습에는 더 많은 에포크가 요구된다. 그러나 전이학습 모델은 특정 계층을 고정하지 않고 모델의 전체 파라미터를 옵티마이저에 전달하여 갱신되도록 열어두는 미세 조정 전략을 택했다. 이때 학습률을 **1e-4**로 낮게 설정함으로써, **ImageNet**으로 학습된 기존의 유용한 가중치가 급격히 훼손되는 것을 방지함과 동시에 현재의 데이터셋인 **10개 클래스**에 잘 최적화되는 시너지 효과를 발생시킬 수 있었다.

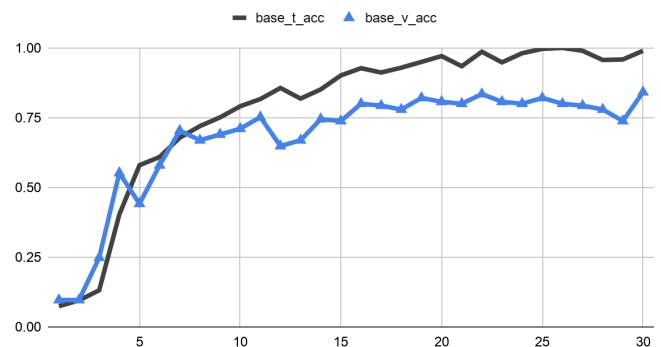
3) train loss와 validation loss를 비교했을 때 과적합이 발생했는가?

30 epoch로 학습을 진행한 베이스 모델()에서, epoch 19에서 Train Loss: 0.1577, Val Loss: 0.7732를 기점으로 지표가 바뀌기 시작하는데, Train Loss (자주색 선)은 0.0860(20 epoch) → 0.0024(26 epoch) → 0.0393(27 epoch)로 점점 줄어드는 추세를 보이거나 Val Loss (초록색 선)은 1.0998(20 epoch) → 1.3172(26 epoch) → 1.5896(27 epoch)로 갑자기 증가하는 추세를 보인다. Train Loss와 Val Loss의 간격이 크게 벌어진 것으로 보아, 훈련 데이터를 과도하게 학습해 검증 데이터에 대한 예측 능력이 떨어지는 과적합이 발생했다.

Base 모델 Loss



베이스 모델



4) augmentation은 성능에 어떤 영향을 주었는가?

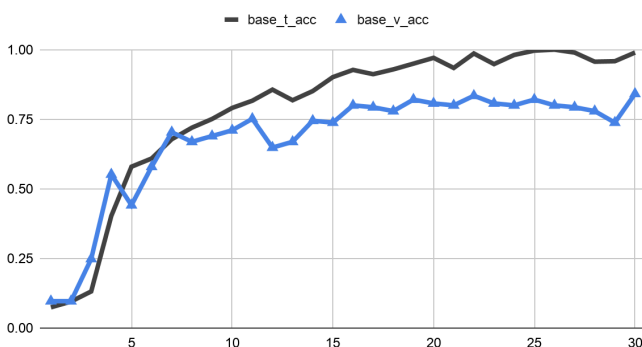
데이터 증강은 데이터셋이 적을때의 오버피팅은 줄이고, 모델의 일반화 성능을 늘리는데 기여한다. 다만 **baseline**모델은 Acc가 0.7724, 증강 모델은 Acc가 0.7310으로 실험결과 기본 CNN모델 보다 성능이 낮게 나왔다. 둘다 train accuracy가 특정 수준으로 수렴하지 않고 있고, val loss와 val acc가 각각 약간의 흔들림은 있지만 감소하고, 상승하는 모습을 보여준다. 또한 데이터를 가공해, 일반화 성능을 높이는 데이터 증강 자체가 많은 학습횟수를 요한다는 점을 감안하면, 낮은 성능은 코드 실행환경에 대한 제한으로 적은 epoch수가 문제라고 여겨진다. 아마 epoch 수를 늘려 비교하면, 정확하게 Augmentation에 대한 유의미한 결과를 얻을 수 있을 것으로 예상 된다.


```
Epoch [1/10]: 100% | 37/37 [03:29:00.00, 5.66s/it, train_acc=0.0900, train_loss=2.3039]
Epoch [1/10]: 100% | 37/37 [03:29:00.00, 5.66s/it, train_acc=0.0900, train_loss=2.3039]
Epoch [2/10]: 100% | 37/37 [00:13:00.00, 2.651t/s, train_acc=0.3080, train_loss=1.8934]
Epoch [2/10]: 100% | 37/37 [00:13:00.00, 2.651t/s, train_acc=0.3080, train_loss=1.8934]
Epoch [3/10]: 100% | 37/37 [00:14:00.00, 2.821t/s, train_acc=0.5415, train_loss=1.3246]
Epoch [3/10]: 100% | 37/37 [00:14:00.00, 2.821t/s, train_acc=0.5415, train_loss=1.3246]
Epoch [4/10]: 100% | 37/37 [00:14:00.00, 2.631t/s, train_acc=0.5900, train_loss=1.1490]
Epoch [4/10]: 100% | 37/37 [00:14:00.00, 2.631t/s, train_acc=0.5900, train_loss=1.1490]
Epoch [5/10]: 100% | 37/37 [00:14:00.00, 2.841t/s, train_acc=0.6055, train_loss=1.1133]
Epoch [5/10]: 100% | 37/37 [00:14:00.00, 2.841t/s, train_acc=0.6055, train_loss=1.1133]
Epoch [6/10]: 100% | 37/37 [00:14:00.00, 2.841t/s, train_acc=0.6055, train_loss=1.1133]
Epoch [6/10]: 100% | 37/37 [00:14:00.00, 2.841t/s, train_acc=0.6055, train_loss=1.1133]
Epoch [7/10]: 100% | 37/37 [00:14:00.00, 2.611t/s, train_acc=0.7612, train_loss=0.6678]
Epoch [7/10]: 100% | 37/37 [00:14:00.00, 2.611t/s, train_acc=0.7612, train_loss=0.6678]
Epoch [8/10]: 100% | 37/37 [00:11:00.00, 3.151t/s, train_acc=0.8114, train_loss=0.5180]
Epoch [8/10]: 100% | 37/37 [00:11:00.00, 3.151t/s, train_acc=0.8114, train_loss=0.5180]
Epoch [9/10]: 100% | 37/37 [00:13:00.00, 2.891t/s, train_acc=0.8564, train_loss=0.4059]
Epoch [9/10]: 100% | 37/37 [00:13:00.00, 2.891t/s, train_acc=0.8564, train_loss=0.4059]
Epoch [10/10]: 100% | 37/37 [00:12:00.00, 2.871t/s, train_acc=0.8720, train_loss=0.3493]
Epoch [10/10]: 100% | 37/37 [00:12:00.00, 2.871t/s, train_acc=0.8720, train_loss=0.3493]
Best Validation Accuracy: 0.7724
Best Validation Accuracy: 0.7724
```

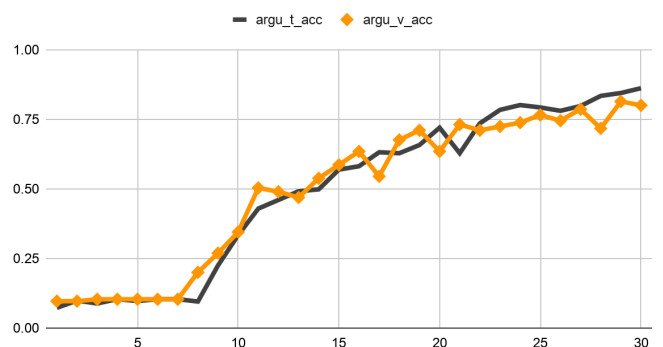
```
Epoch [1/10]: 100% | 37/37 [00:13:00.00, 2.671t/s, train_acc=0.0969, train_loss=2.3060]
Epoch [1/10]: 100% | 37/37 [00:13:00.00, 2.671t/s, train_acc=0.0969, train_loss=2.3060]
Epoch [2/10]: 100% | 37/37 [00:14:00.00, 2.631t/s, train_acc=0.1817, train_loss=2.2008]
Epoch [2/10]: 100% | 37/37 [00:14:00.00, 2.631t/s, train_acc=0.1817, train_loss=2.2008]
Epoch [3/10]: 100% | 37/37 [00:14:00.00, 2.581t/s, train_acc=0.3270, train_loss=1.8583]
Epoch [3/10]: 100% | 37/37 [00:14:00.00, 2.581t/s, train_acc=0.3270, train_loss=1.8583]
Epoch [4/10]: 100% | 37/37 [00:16:00.00, 2.281t/s, train_acc=0.4732, train_loss=1.5160]
Epoch [4/10]: 100% | 37/37 [00:16:00.00, 2.281t/s, train_acc=0.4732, train_loss=1.5160]
Epoch [5/10]: 100% | 37/37 [00:13:00.00, 2.691t/s, train_acc=0.4965, train_loss=1.3637]
Epoch [5/10]: 100% | 37/37 [00:13:00.00, 2.691t/s, train_acc=0.4965, train_loss=1.3637]
Epoch [6/10]: 100% | 37/37 [00:15:00.00, 2.451t/s, train_acc=0.5969, train_loss=1.1519]
Epoch [6/10]: 100% | 37/37 [00:15:00.00, 2.451t/s, train_acc=0.5969, train_loss=1.1519]
Epoch [7/10]: 100% | 37/37 [00:13:00.00, 2.681t/s, train_acc=0.6678, train_loss=0.9255]
Epoch [7/10]: 100% | 37/37 [00:13:00.00, 2.681t/s, train_acc=0.6678, train_loss=0.9255]
Epoch [8/10]: 100% | 37/37 [00:14:00.00, 2.591t/s, train_acc=0.6713, train_loss=0.9266]
Epoch [8/10]: 100% | 37/37 [00:14:00.00, 2.591t/s, train_acc=0.6713, train_loss=0.9266]
Epoch [9/10]: 100% | 37/37 [00:14:00.00, 2.551t/s, train_acc=0.6955, train_loss=0.8635]
Epoch [9/10]: 100% | 37/37 [00:14:00.00, 2.551t/s, train_acc=0.6955, train_loss=0.8635]
Epoch [10/10]: 100% | 37/37 [00:14:00.00, 2.631t/s, train_acc=0.7180, train_loss=0.7921]
Epoch [10/10]: 100% | 37/37 [00:14:00.00, 2.631t/s, train_acc=0.7180, train_loss=0.7921]
Best Validation Accuracy: 0.7310
Best Validation Accuracy: 0.7310
```

따라서 30epoch로 늘려 실험한 결과는 5.의 표와 같다. **Best Validation Accuracy**는 기본 모델이 0.8414로 0.8138인 데이터 증강 모델보다 우수하지만, 아래의 그래프를 보면

베이스 모델



데이터 증강 모델



베이스 모델은 비교적 빠른 속도로 수렴한다. 에포크 15 부근에서 훈련 정확도(Train Acc) 0.9를 돌파하며, 최종적으로 0.9896에 도달했으며, 훈련 데이터에 거의 완벽히 맞춰졌다. 반면 데이터 증강 모델은 비교적 완만하고, 천천히 수렴한다. 에포크 30 기준 최종 훈련 정확도는 0.8616이며, 에포크 수를 늘려 더 학습을 진행할 시에 모델 성능이 향상될 여지가 존재한다.

베이스 모델의 경우 Train Loss는 지속해서 감소(최종 0.0402)하나, Val Loss는 에포크 20 이후 1.0 이상으로 치솟는 구간(에포크 26: 1.3172, 에포크 27: 1.5896)이 존재하여 지표의 변동성이 크다. 데이터 증강 모델의 경우 과적합 징후가 현저히 적고 안정적이다. Train Loss와 Val Loss이 동반 하락하며, 훈련 정확도와 검증 정확도 간의 격차가 작다. 최종 에포크 부근에서 가장 낮고 안정적인 검증 손실(0.6745)을 기록했다.

즉 데이터 증강은 모델의 수렴 속도를 늦추고, 비교적 **Best Validation Accuracy**를 낮추는 결과를 가져왔으나, 과적합 문제를 완화하고, 학습 곡선 안정화에 기여했으며, 더 많이 학습을 진행할 경우 데이터 증강 모델의 최종 성능이 베이스 모델을 뛰어넘을 가능성이 높다.

5) pretrained 사용 여부는 어떤 차이를 만들었는가?

pretrained를 사용한 모델과 그렇지 않은 **Baseline** 모델은 학습 속도(정확도)와 **Overfitting** 방지에서 차이를 보였다.

- 먼저, 처음부터 100% 정답을 향해 빠르게 학습할 수 있었다. **Baseline** 모델은 이미지에서 선이나 모양을 찾는 기초적인 방법부터 바닥부터 배워야 했지만 pretrained 모델은 이미

엄청나게 많은 사진을 보고 사물을 구별하는 요령을 아는 상태에서 시작했다. 덕분에 기초 학습 과정을 건너뛰고, 불과 15 에포크라는 짧은 시간 안에 100%라는 완벽한 정확도에 도달할 수 있었다.

- 다음으로, 과적합이 방지되었다. 데이터가 제한적일 때 인공지능은 정답을 맞추기 위해 배경 노이즈 같은 쓸데없는 부분까지 통째로 외워버리는 문제를 일으킨다. 하지만 pretrained 모델은 이미 세상의 일반적인 시각적 형태를 많이 알고 있기 때문에, 우리 훈련 데이터가 적더라도 과적합과 같은 함정에 빠지지 않고 새로운 이미지의 핵심 패턴을 잘 짚어낼 수 있었다.

6) 성능 개선을 위해 추가로 시도할 수 있는 방법은 무엇인가?

723개의 비교적 적은 데이터로 10개의 클래스를 분류해야 하므로, 모델이 충분한 feature를 학습하기에 데이터셋이 절대적으로 부족한 상태이다. 앞선 데이터 증강 모델 실험에서 모델의 성능이 완만하게 지속적으로 상승하는 양상이 관찰되며, 이는 모델이 학습할 데이터가 많고, 다양해진다면 성능 향상의 여지가 있음을 시사한다.

7. 팀원별 기여도

이름	담당 역할	실제 수행 내용	이해한 핵심 개념
이지현	A	데이터 구조 분석, CSV 처리, Dataset/DataLoader 구현, 데이터 증강 실험, 보고서 1., 2., 3., 5., 6.1),3),4),6) 작성	데이터 증강이 실제로 어떻게 적용되는지, 왜 증강 후의 image 수가 예상과 달리 증강 전과 동일한지 알 수 있었다.
최유경	B	CNN model 부분 및 하이퍼파라미터 관련 실험 파이프라인 구현	학습 과정을 코드로 구현하며 실제로 어떤 단계를 거치며 학습이 일어나는지 더 잘 파악할 수 있게 되었다.
박소현	C	전이 학습 코드 작성 및 관련 부분 보고서 작성	우선 이론으로만 공부한 것들로 실습을 처음 해봐서 생각보다 더 어려웠고, 내 파트인 “전이학습”의 단계 구현에 대해서만 이해하게 되어 아쉽다. 다음 실습때는 전이학습이 아닌 다른 단계를 담당하여, 실습에 관한 스킬을 더 늘리고 싶다.

8. 생성형 AI 사용 기록

[지현]

- 드라이브 마운트 관련

Google colab으로 협업을 진행 해본적이 없어, 다른 분들의 공유 폴더에서도 코드가 정상 작동하도록 하는게 중요했다. 실제 코랩내부의 경로가 어떻게 설정되는지 몰라 이와 관련해 생성형 ai를 썼다

질문 : googled colab 내부에서 협업진행, 구글 드라이브를 통해서 파일을 공유 중. 코드와 같은 경로에 **dataset/**폴더가 존재한다. **dataset** 내부엔 각각의 이미지를 가진 폴더 **train/**, **test/**이 존재하며, 이미지에대한 메타데이터가 담긴 **train.csv**, **test.csv** 파일이 위치하며. 드라이브 연동 방법 요청.

- random seed 관련

모델 학습시, 동일 모델이어도 언제 테스트 했냐에 따라 상이한 결과가 도출되었다.

하이퍼파라미터 실험에서 베이스모델과 동일한 **optimizer**, **lr**을 이용했음에도 **Best Validation Accuracy**이 달라, 실험에서 확인하고자하는 결과를 보기에 어려움이 존재했다. 아마 임의의 값으로 초기 가중치가 주어진다는 점과 테스트 이미지의 **Shuffle**로 발생하는 차이로 예상되어 이를 해결하기 위한 방법을 요청했다.

[유경]

- 모델 구축 관련

기존 수업 등에서 **cnn** 구현해놓은 코드가 있어서 해당 코드와 협업 진행중인 파일을 같이 넣고 해당 데이터와 데이터로더에 맞게 학습을 진행할 수 있도록 진행하기 위해 **ai**를 사용했다.

- 하이퍼파라미터 실험 관련

구축한 모델 코드 내부에서 하이퍼파라미터를 하나하나 변경하면서 실험하기는 귀찮아서 외부 인자로 받고 **train**을 진행할 수 있도록 설계하기 위한 로직을 요청했다.

[소현]

- 전체적인 흐름 파악 & 전이학습 코드 설계

지현님과 유경님이 보내주시는 코드들이 전부 처음 보는 코드여서, 어떤 코드인지 확인하고 공부하기 위해 **AI**에게 **.ipynb**로 파일을 다운로드하여 설명해달라고 부탁했다. 또, 이렇게 파악한 코드를 기반으로 전이학습 코드를 구현해달라고 요청했다. 다만 여기서 한 가지 아쉬운 점은, 이론을 공부했지만 막상 실습에 적용하려니 혼란스럽거나 잘 모르겠는 부분이 많아서, 방학 때 이론 역시 더욱 공부하며 보강해야 겠다는 생각이 들었다.